

E-Commerce Product Catalog: Mock Interview Preparation

Project Overview

This project is a Java 17 Maven console application for an **E-Commerce Product Catalog**. It is designed to demonstrate important Java and object-oriented programming concepts using a simple product catalog domain.

The project models different product types such as electronics, clothing, and books. Each product has shared behavior from an abstract base class, while product-specific behavior is implemented through subclasses and interfaces.

Main project flow:

1. `pom.xml` configures the Maven project and Java version.
2. `ECommerceCatalogApp` starts the application.
3. Sample products are created in memory.
4. Products are stored in `ProductCatalogService`.
5. The app demonstrates OOP, records, static members, nested classes, lambdas, streams, filtering, sorting, shipping, discounts, reviews, and return policies.

1. Entire Workflow And Working Of The Project

1. Maven Configuration

File: `pom.xml`

The `pom.xml` file defines the Maven project.

Important configuration:

- `groupId: com.ecommerce`
- `artifactId: product-catalog`
- `version: 1.0.0`
- `Java version: 17`
- `Main class: com.ecommerce.catalog.ECommerceCatalogApp`

The project uses:

- maven-compiler-plugin to compile Java 17 code.
- exec-maven-plugin to run the console application from Maven.

This file is responsible for build and execution configuration.

2. Application Entry Point

File: src/main/java/com/ecommerce/catalog/ECommerceCatalogApp.java

This is the main class of the project.

Execution starts from:

```
public static void main(String[] args)
```

The main workflow is:

1. Create a sample catalog using `buildSampleCatalog()`.
2. Print the full catalog.
3. Demonstrate static discount rules and final tax rates.
4. Demonstrate the static nested class `Product.CategoryUtils`.
5. Demonstrate polymorphism and method overriding.
6. Demonstrate records using product reviews and shipping details.
7. Demonstrate interface-based capabilities.
8. Demonstrate lambda search, filtering, and sorting.

The `buildSampleCatalog()` method creates:

- Electronics laptop
- Clothing jacket
- Book novel
- Electronics headphones

It also adds reviews to some products using the `ProductReview` record.

This class acts like the demo runner. It connects all model, interface, record, and service classes together.

3. Abstract Base Class

File: src/main/java/com/ecommerce/catalog/model/Product.java

`Product` is an abstract class. It represents common data and behavior shared by all product types.

Common fields:

- sku
- name
- basePrice
- brand
- reviews

These fields are shared because every product in the catalog needs basic identity, pricing, brand, and review information.

Important abstract methods:

```
public abstract String getCategoryKey();
public abstract String getProductTypeDescription();
public abstract BigDecimal calculateFinalPrice(BigDecimal discountedPrice);
```

These methods are abstract because each product type has different category information, display description, and tax behavior.

Shared methods:

- addReview(ProductReview review)
- getReviews()
- getAverageRating()
- applyTax(BigDecimal amount, BigDecimal taxRate)
- toString()
- getter methods for product fields

The class also contains static discount rules:

```
public static final Map<String, BigDecimal> CATEGORY_DISCOUNT_RULES;
```

Category discounts:

- ELECTRONICS: 10 percent
- CLOTHING: 15 percent
- BOOK: 5 percent

It also contains final tax rate constants:

- STANDARD_TAX_RATE
- REDUCED_TAX_RATE
- LUXURY_TAX_RATE

The nested static class CategoryUtils provides helper methods:

- `isValidCategory(String categoryKey)`
- `formatCategoryLabel(String categoryKey)`
- `resolveDiscountFor(Product product)`

Interview explanation:

I used an abstract Product class because all products share common fields and behavior, but each category needs to provide its own implementation for category, description, and final price calculation.

4. Electronics Product

File: `src/main/java/com/ecommerce/catalog/model/Electronics.java`

Electronics extends Product.

It implements:

- `Discountable`
- `Shippable`
- `Returnable`

Extra fields:

- `warrantyMonths`
- `fragile`
- `activeDiscountPercent`

Important behavior:

- Category key is `ELECTRONICS`.
- Product description includes warranty and fragile information.
- Final price uses `LUXURY_TAX_RATE`.
- Discount is calculated from `activeDiscountPercent`.
- Shipping cost depends on whether the product is fragile.
- International shipping adds extra cost.
- Return window is 30 days.

This is the best class to explain multiple interface implementation because it supports discounting, shipping, and returns.

Interview explanation:

Electronics is a product with all major capabilities. It can be discounted, shipped, and returned. This shows how interfaces can be combined to model real-world product behavior.

5. Clothing Product

File: `src/main/java/com/ecommerce/catalog/model/Clothing.java`

Clothing extends Product.

It implements:

- Discountable
- Returnable

Extra fields:

- size
- material
- activeDiscountPercent

Important behavior:

- Category key is CLOTHING.
- Product description includes size and material.
- Final price uses STANDARD_TAX_RATE.
- Return window is 45 days.
- It does not implement Shippable.
- It has a helper method called `getStandardApparelShipping()`.

This shows that a class should only implement interfaces that match its required behavior.

Interview explanation:

Clothing is discountable and returnable, but in this demo it does not implement the Shippable interface. This shows that interfaces give flexibility instead of forcing every product to support every feature.

6. Book Product

File: `src/main/java/com/ecommerce/catalog/model/Book.java`

Book extends Product.

It implements:

- Discountable
- Shippable

Extra fields:

- author
- isbn
- pageCount
- activeDiscountPercent

Important behavior:

- Category key is BOOK.
- Product description includes author, page count, and ISBN.
- Final price uses REDUCED_TAX_RATE.
- Shipping uses Media Mail.
- Shipping cost depends on page count.
- It does not implement Returnable.

Interview explanation:

The Book class shows category-specific behavior. It has a reduced tax rate and a custom shipping method, but it is not returnable in this demo.

7. Interfaces

Interfaces are stored in:

- src/main/java/com/ecommerce/catalog/interfaces/Discountable.java
- src/main/java/com/ecommerce/catalog/interfaces/Shippable.java
- src/main/java/com/ecommerce/catalog/interfaces/Returnable.java

Discountable

Discountable represents products that support discounts.

Methods:

- getBasePrice()
- applyDiscount(BigDecimal discountPercent)
- getDiscountedPrice()

Implemented by:

- Electronics
- Clothing

- Book

Shippable

Shippable represents products that can calculate shipping.

Methods:

- calculateShipping(String destinationCountry)
- getEstimatedDeliveryDays()
- requiresSpecialHandling()

Implemented by:

- Electronics
- Book

Returnable

Returnable represents products that can be returned.

Methods:

- getReturnWindowDays()
- isReturnEligible(LocalDate purchaseDate)
- getReturnPolicySummary()

Implemented by:

- Electronics
- Clothing

Interview explanation:

Interfaces are used to model optional capabilities. Not every product needs shipping, returns, or discounts, so interfaces keep the design flexible and clean.

8. Records

Records are stored in:

- src/main/java/com/ecommerce/catalog/records/ProductReview.java
- src/main/java/com/ecommerce/catalog/records/ShippingDetails.java

ProductReview

ProductReview is an immutable record for customer review data.

Fields:

- reviewerName
- rating
- comment
- reviewDate

It has a compact constructor that validates the rating:

```
if (rating < 1 || rating > 5) {  
    throw new IllegalArgumentException("Rating must be between 1 and 5");  
}
```

This ensures rating is always between 1 and 5.

ShippingDetails

ShippingDetails is an immutable record for shipping quote data.

Fields:

- carrier
- shippingCost
- estimatedDays
- trackingIncluded

Interview explanation:

I used records because reviews and shipping details are simple immutable data carriers. Records reduce boilerplate by automatically generating constructor, accessor methods, equals, hashCode, and toString.

9. Product Catalog Service

File: src/main/java/com/ecommerce/catalog/service/ProductCatalogService.java

This class manages catalog operations.

It stores products in memory:

```
private final List<Product> catalog = new ArrayList<>();
```

Main responsibilities:

- Add products

- Return all products safely
- Search by name
- Filter by category
- Filter by price
- Filter by rating
- Sort by price
- Find products using custom predicates
- Get products based on implemented interfaces
- Print catalog summaries
- Print product reviews

Important methods:

- `addProduct(Product product)`
- `getAllProducts()`
- `searchByName(String keyword)`
- `filterByCategory(String categoryKey)`
- `filterByMaxPrice(BigDecimal maxPrice)`
- `filterByMinAverageRating(double minRating)`
- `filter(Predicate<Product> criteria)`
- `findProductsMatching(Predicate<Product>... predicates)`
- `getDiscountableProducts()`
- `getShippableProducts()`
- `getReturnableProducts()`
- `sortByPriceAscending()`
- `printCatalogSummary()`
- `printReviewsFor(Product product)`

The service uses Java streams and lambdas for filtering and sorting.

Example:

```
return catalog.stream()
    .filter(p -> p.getName().toLowerCase().contains(lower))
    .collect(Collectors.toList());
```

The service also protects internal data:

```
return List.copyOf(catalog);
```

This prevents callers from directly modifying the internal catalog list.

Interview explanation:

ProductCatalogService separates catalog operations from the model classes. It manages the collection of products and provides search, filter, sort, and capability-based lookup operations.

2. Important Topics Covered In This Project

Object-Oriented Programming

This project strongly demonstrates OOP.

Covered concepts:

- Class
- Object
- Abstract class
- Inheritance
- Method overriding
- Polymorphism
- Encapsulation
- Interface-based design

Example:

```
Product product = new Electronics(...);
```

Even though the reference type is `Product`, the actual object is `Electronics`. Java calls overridden methods based on the runtime object.

Abstract Class

`Product` is abstract because it should not be created directly.

It provides:

- Shared fields
- Shared methods
- Required methods for subclasses

Each subclass must implement its own category, description, and price calculation logic.

Inheritance

Inheritance is shown through:

```
public class Electronics extends Product
public class Clothing extends Product
public class Book extends Product
```

These classes reuse common behavior from Product.

Polymorphism

Polymorphism appears when the catalog stores products as Product references:

```
for (Product product : catalog.getAllProducts()) {  
    BigDecimal finalPrice = product.calculateFinalPrice(discounted);  
}
```

At runtime, Java decides whether to call the Electronics, Clothing, or Book version of calculateFinalPrice().

Interfaces

Interfaces define capabilities:

- Discountable
- Shippable
- Returnable

This allows each product class to implement only the behavior it needs.

Multiple Inheritance Through Interfaces

Java does not allow a class to extend multiple classes, but it allows a class to implement multiple interfaces.

Example:

```
public class Electronics extends Product  
    implements Discountable, Shippable, Returnable
```

This means Electronics can act as a product, a discountable item, a shippable item, and a returnable item.

Java Records

Records are used for immutable data classes.

Used records:

- ProductReview
- ShippingDetails

Records are useful because they reduce boilerplate and are ideal for simple data objects.

Encapsulation

The project protects internal state by:

- Keeping fields private or protected.
- Providing getter methods.
- Returning unmodifiable reviews.
- Returning a defensive copy of the catalog.

Example:

```
return Collections.unmodifiableList(reviews);
```

BigDecimal For Money

Prices, discounts, taxes, and shipping costs use `BigDecimal`.

This is important because money calculations should avoid floating-point precision issues.

Example problem with `double`:

```
0.1 + 0.2
```

This may not produce exactly `0.3` due to binary floating-point representation.

`BigDecimal` avoids that problem when used correctly.

Static Members

Static members belong to the class, not to individual objects.

In this project:

```
public static final Map<String, BigDecimal> CATEGORY_DISCOUNT_RULES;
```

All products share the same category discount rules.

Final Constants

Tax rates are declared as final constants:

```
public static final BigDecimal STANDARD_TAX_RATE
public static final BigDecimal REDUCED_TAX_RATE
public static final BigDecimal LUXURY_TAX_RATE
```

This shows reusable fixed business rules.

Static Nested Class

`Product.CategoryUtils` is a static nested utility class.

It is used for category-related helper methods:

- Validate category
- Format category label
- Resolve category discount

Lambdas And Streams

The service layer uses streams and lambdas for clean data processing.

Examples:

- Search by name
- Filter by category
- Filter by max price
- Filter by minimum average rating
- Sort by price
- Combine multiple predicates

Interview explanation:

Instead of writing manual loops everywhere, I used streams to process the product list in a readable and functional style.

Predicate

`Predicate<Product>` is used for flexible filtering.

Example:

```
p -> p.getBasePrice().compareTo(new BigDecimal("50")) > 0
```

The method `findProductsMatching()` combines multiple predicates using `Predicate::and`.

Pattern Matching With instanceof

The app uses:

```
product instanceof Discountable d
```

This checks whether a product implements `Discountable` and directly creates a typed variable `d`.

This is cleaner than old-style casting.

Date Handling

The project uses `LocalDate` for:

- Review dates
- Return eligibility checks

Example:

```
LocalDate.now().minusDays(getReturnWindowDays())
```

This checks whether a purchase is still inside the return window.

3. Top 10 Important Interview Questions

1. Why did you make Product an abstract class?

Expected answer:

Product is abstract because it contains common fields and behavior for all products, but it should not be instantiated directly. Every product must provide its own category, description, and final price calculation, so those methods are abstract.

2. How is polymorphism used in this project?

Expected answer:

The catalog stores different product types as Product references. During execution, Java calls the overridden method of the actual runtime object, such as Electronics, Clothing, or Book.

Example:

```
product.calculateFinalPrice(discountedPrice)
```

The actual calculation depends on the product subclass.

3. Why did you use interfaces like Discountable, Shippable, and Returnable?

Expected answer:

Interfaces are used because not every product supports the same behavior. Some products are shippable, some are returnable, and some are discountable. Interfaces allow each class to implement only the capabilities it needs.

4. How does this project show multiple inheritance?

Expected answer:

Java does not support multiple class inheritance, but a class can implement multiple interfaces. Electronics extends Product and implements Discountable, Shippable, and Returnable, so it demonstrates multiple inheritance through interfaces.

5. Why did you use records for ProductReview and ShippingDetails?

Expected answer:

Records are best for immutable data carrier classes. ProductReview and ShippingDetails only store data, so records reduce boilerplate and automatically provide constructors, accessors, equals, hashCode,

and toString.

6. Why is BigDecimal better than double for prices?

Expected answer:

BigDecimal is better for money because it avoids floating-point precision errors. In e-commerce, prices, discounts, taxes, and shipping costs must be accurate.

7. What is the role of ProductCatalogService?

Expected answer:

ProductCatalogService is the service layer. It manages the in-memory product list and provides operations like adding products, searching, filtering, sorting, and finding products by interface capability.

8. How are lambdas and streams used in this project?

Expected answer:

They are used in ProductCatalogService to search, filter, and sort products. For example, searchByName() uses a stream and lambda to find products whose names contain a keyword.

9. What is defensive copying, and where is it used?

Expected answer:

Defensive copying prevents outside code from modifying internal data structures. In this project, getAllProducts() returns List.copyOf(catalog) so callers cannot directly change the internal catalog list.

10. How would you add a new product type, such as Furniture?

Expected answer:

I would create a Furniture class that extends Product, override the abstract methods, add furniture-specific fields, implement relevant interfaces like Shippable or Returnable, add category discount rules if needed, and add sample furniture products to ECommerceCatalogApp.

Bonus Questions To Practice

What happens if a product has no reviews?

getAverageRating() returns 0.0 because the stream average uses:

```
.orElse(0.0)
```

Why does ProductReview validate rating in the constructor?

It protects the object from invalid data. A review rating should always be between 1 and 5.

Why does Product use Objects.requireNonNull()?

It prevents required fields like SKU, name, base price, and brand from being null.

What is the difference between getReviews() and directly exposing the review list?

getReviews() returns an unmodifiable list, so outside code can read reviews but cannot directly modify the internal list.

Why is CategoryUtils static?

It does not need object state. It only provides helper methods related to product categories, so making it static is appropriate.

Why does findProductsMatching() use varargs?

It allows callers to pass any number of filtering conditions.

Example:

```
catalog.findProductsMatching(  
    p -> p.getBasePrice().compareTo(new BigDecimal("50")) > 0,  
    p -> p.getCategoryKey().equals("ELECTRONICS")  
);
```

This returns products that satisfy all given conditions.

Suggested 2-Minute Project Explanation

This is a Java 17 console-based e-commerce product catalog project. I built it mainly to demonstrate object-oriented programming concepts in Java.

The base class is Product, which is abstract because every product shares common data like SKU, name, price, brand, and reviews, but each product type has its own category, description, and final price calculation. Then I created concrete product classes: Electronics, Clothing, and Book.

I used interfaces like Discountable, Shippable, and Returnable to model optional capabilities. For example, electronics support all three, clothing supports discounts and returns, and books support discounts and shipping. This keeps the design flexible.

I also used Java records for immutable data objects like `ProductReview` and `ShippingDetails`. The service layer, `ProductCatalogService`, manages products in memory and uses streams, lambdas, and predicates for searching, filtering, sorting, and capability-based lookup.

Overall, the project shows abstraction, inheritance, polymorphism, encapsulation, interfaces, records, static members, nested classes, and functional programming features in Java.

Key Files To Mention In Interview

- `pom.xml`: Maven and Java 17 configuration.
- `ECommerceCatalogApp.java`: Main application runner.
- `Product.java`: Abstract base class for all products.
- `Electronics.java`: Product with discount, shipping, and return behavior.
- `Clothing.java`: Product with discount and return behavior.
- `Book.java`: Product with discount and shipping behavior.
- `Discountable.java`: Interface for discount behavior.
- `Shippable.java`: Interface for shipping behavior.
- `Returnable.java`: Interface for return behavior.
- `ProductReview.java`: Record for customer reviews.
- `ShippingDetails.java`: Record for shipping quotes.
- `ProductCatalogService.java`: Service layer for catalog operations.

Final Interview Focus

Be ready to explain:

1. Why `Product` is abstract.
2. How subclasses override product behavior.
3. How interfaces model flexible capabilities.
4. How records simplify immutable data classes.
5. Why `BigDecimal` is used for money.
6. How streams and lambdas make filtering cleaner.
7. How defensive copying protects internal state.
8. How the project could be extended with a new product type.